



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

Лабораторная работа № 7

на тему:

«Игры на графах. Производящая функция и вектор Майерсона»

Вариант 13

Студент:

ИУ8-105
(Группа)

Б. Е. Шекунов
(И. О. Фамилия)

Преподаватель:

Н. С. Коннова
(И. О. Фамилия)

Москва, 2026 г.

Цель работы:

Исследовать кооперативную игру на графе, построить производящую функцию для выбранных вершин и вычислить соответствующие компоненты вектора Майерсона.

Постановка задачи:

В работе рассматривается неориентированный граф, в котором вершины соответствуют игрокам, а рёбра – возможным прямым связям между ними.

Для выбранных вершин графа требуется:

1. построить производящую функцию $\varphi_p(x)$;
2. вычислить компоненту вектора Майерсона $Y_p(v, g)$;

Исходные данные:

Задан тестовый граф на 8 вершинах:

$$E = \{(1,2), (2,3), (3,4), (1,5), (2,6), (5,7), (5,8)\}.$$

Граф является деревом.

Вершина 1 выбрана для подробного анализа.

Тестовый граф. Выделена вершина 1

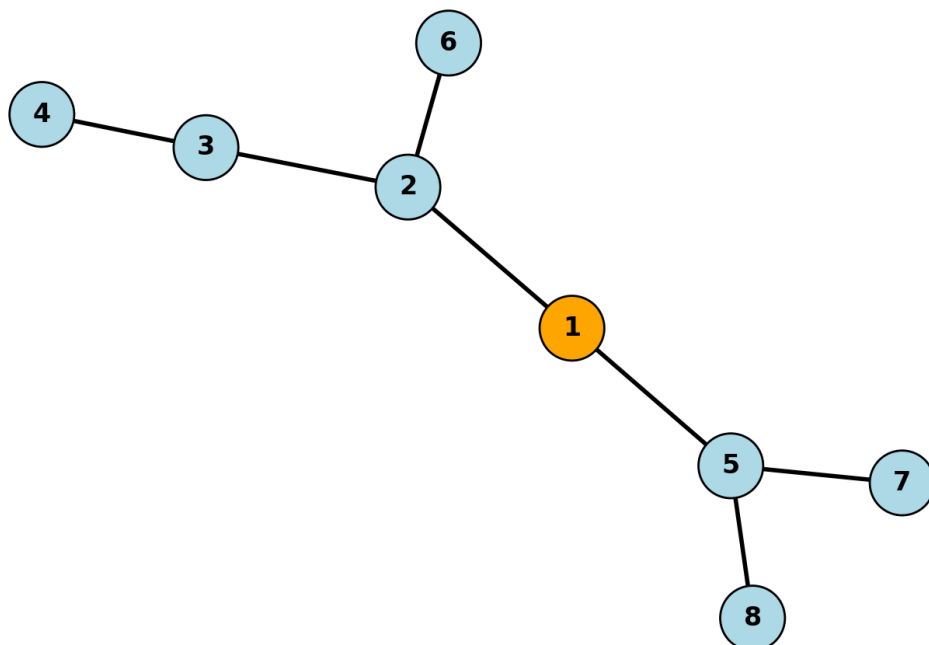


Рисунок 1 – Тестовый граф

Ход работы:

Пусть задан граф:

$$g = (N, E),$$

где N – множество вершин, E – множество рёбер.

Для вершины p вводится производящая функция:

$$\varphi_p(x) = \sum_m a_m x^m,$$

Где a_m – количество простых путей, содержащих вершину p и состоящих из m вершин.

Если путь содержит m вершин, то его длина равна:

$$k = m - 1.$$

Вектор Майерсона для вершины p вычисляется по формуле:

$$Y_p(v, g) = \sum_{k \geq 1} \frac{A_k}{k+1} r^k.$$

где A_k — число путей длины k рёбер, содержащих вершину p .

Каждый путь даёт вклад r^k , который делится поровну между всеми m вершинами, входящими в путь.

Для вершины $p = 1$ граф разбивается на две ветви: через вершины 2 и 5.

Рекурсивный расчёт производящих функций ветвей:

$$\psi_4 = x,$$

$$\psi_6 = x,$$

$$\psi_7 = x,$$

$$\psi_8 = x,$$

$$\psi_3 = x(1 + \psi_4) = x + x^2,$$

$$\psi_2 = x(1 + \psi_3 + \psi_6) = x + 2x^2 + x^3,$$

$$\psi_5 = x(1 + \psi_7 + \psi_8) = x + 2x^2.$$

Тогда производящая функция для вершины 1:

$$\varphi_1(x) = x(1 + \psi_2 + \psi_5 + \psi_2\psi_5).$$

После раскрытия скобок получено:

$$\varphi_1(x) = x + 2x^2 + 5x^3 + 5x^4 + 5x^5 + 2x^6.$$

Отсюда:

$$a_1 = 1, a_2 = 2, a_3 = 5, a_4 = 5, a_5 = 5, a_6 = 2.$$

Коэффициенты для путей длины k рёбер:

$$A_1 = 2, A_2 = 5, A_3 = 5, A_4 = 5, A_5 = 2.$$

Компонента вектора Майерсона для вершины 1:

$$Y_1(v, g) = r + \frac{5}{3}r^2 + \frac{5}{4}r^3 + r^4 + \frac{1}{3}r^5.$$

Результаты для остальных вершин

Вершина 2:

$$\varphi_2(x) = x + 3x^2 + 5x^3 + 6x^4 + 5x^5 + 2x^6$$

$$Y_2(v, g) = \frac{3}{2}r + \frac{5}{3}r^2 + \frac{3}{2}r^3 + r^4 + \frac{1}{3}r^5$$

Вершина 3:

$$\varphi_3(x) = x + 2x^2 + 3x^3 + 3x^4 + 3x^5 + 2x^6$$

$$Y_3(v, g) = r + r^2 + \frac{3}{4}r^3 + \frac{3}{5}r^4 + \frac{1}{3}r^5$$

Вершина 4:

$$\varphi_4(x) = x + x^2 + x^3 + 2x^4 + x^5 + 2x^6$$

$$Y_4(v, g) = \frac{1}{2}r + \frac{1}{3}r^2 + \frac{1}{2}r^3 + \frac{1}{5}r^4 + \frac{1}{3}r^5$$

Вершина 5:

$$\varphi_5(x) = x + 3x^2 + 4x^3 + 4x^4 + 5x^5 + 2x^6$$

$$Y_5(v, g) = \frac{3}{2}r + \frac{4}{3}r^2 + r^3 + r^4 + \frac{1}{3}r^5$$

Вершина 6:

$$\varphi_6(x) = x + x^2 + 2x^3 + 2x^4 + 2x^5$$

$$Y_6(v, g) = \frac{1}{2}r + \frac{2}{3}r^2 + \frac{1}{2}r^3 + \frac{2}{5}r^4$$

Вершина 7:

$$\varphi_7(x) = x + x^2 + 2x^3 + x^4 + 2x^5 + x^6$$
$$Y_7(v, g) = \frac{1}{2}r + \frac{2}{3}r^2 + \frac{1}{4}r^3 + \frac{2}{5}r^4 + \frac{1}{6}r^5$$

Вершина 8:

$$\varphi_8(x) = x + x^2 + 2x^3 + x^4 + 2x^5 + x^6$$
$$Y_8(v, g) = \frac{1}{2}r + \frac{2}{3}r^2 + \frac{1}{4}r^3 + \frac{2}{5}r^4 + \frac{1}{6}r^5$$

Наибольшее значение производящей функции и наибольший вклад по Майерсону имеют вершины 1, 2 и 5. Это объясняется их положением в графе: через эти вершины проходит большее количество путей между другими вершинами.

Вершины 7 и 8 имеют одинаковые результаты, так как они симметрично расположены относительно вершины 5. Обе являются висячими вершинами, соединёнными с графом через одну и ту же вершину.

Вывод:

В ходе работы была построена производящая функция для каждой вершины заданного дерева и вычислены соответствующие компоненты вектора Майерсона. Для вершины 1 дополнительно выполнен подробный рекурсивный расчёт, который совпал с результатом прямого перебора путей.

Проверка показала, что граф является деревом, поэтому рекурсивный метод применим. Полученные результаты подтверждают, что вершины, находящиеся ближе к центру графа и соединяющие несколько ветвей, имеют больший вклад в кооперативной игре на графе.

Приложение (исходный код)

```
from __future__ import annotations
from collections import defaultdict, deque
from dataclasses import dataclass
from fractions import Fraction
from itertools import combinations
import random
from typing import Dict, Hashable, List, Mapping, Tuple
import matplotlib.pyplot as plt
import networkx as nx

Vertex = Hashable

def fmt_fraction(fr: Fraction) -> str:
    return str(fr.numerator) if fr.denominator == 1 else f"{fr.numerator}/{fr.denominator}"

def format_producer(coeffs: Mapping[int, int], var: str = "x", phi_name: str = "φ") -> str:
    if not coeffs:
        return f"{phi_name}({var}) = 0"

    pieces: list[str] = []
    for power in sorted(coeffs):
        coef = coeffs[power]
        if coef == 0:
            continue

        if power == 0:
            monomial = "1"
        elif power == 1:
            monomial = var
        else:
            monomial = f"{var}^{power}"

        if coef == 1 and power != 0:
            pieces.append(monomial)
        else:
            pieces.append(f"{coef}{monomial} if power != 0 else ''")

    return f"{phi_name}({var}) = " + " + ".join(pieces)

def format_myerson(terms: Mapping[int, Fraction], vertex: Vertex, var: str = "r") -> str:
    if not terms:
        return f"Y_{vertex}(v,g) = 0"

    pieces: list[str] = []
    for degree in sorted(terms):
        coef_s = fmt_fraction(terms[degree])
        monomial = var if degree == 1 else f"{var}^{degree}"
        pieces.append(f"{coef_s}*{monomial}")

    return f"Y_{vertex}(v,g) = " + " + ".join(pieces)

@dataclass(frozen=True)
class UndirectedGraph:
    edges: Tuple[Tuple[Vertex, Vertex], ...]

    def __post_init__(self) -> None:
        normalized: list[tuple[Vertex, Vertex]] = []
        for u, v in self.edges:
            if u == v:
                raise ValueError(f"Петли не поддерживаются: ({u}, {v})")
            normalized.append((u, v))
        object.__setattr__(self, "edges", tuple(normalized))

    @property
    def vertices(self) -> Tuple[Vertex, ...]:
        vertices: set[Vertex] = set()
        for u, v in self.edges:
            vertices.add(u)
            vertices.add(v)
        return tuple(sorted(vertices, key=str))

    @property
    def adjacency(self) -> Dict[Vertex, List[Vertex]]:
        adj: dict[Vertex, list[Vertex]] = defaultdict(list)
```

```

    for u, v in self.edges:
        adj[u].append(v)
        adj[v].append(u)
    for vertex in adj:
        adj[vertex] = sorted(adj[vertex], key=str)
    return dict(adj)

def is_tree(self) -> bool:
    vertices = self.vertices
    if not vertices:
        return True
    if len(self.edges) != len(vertices) - 1:
        return False

    visited: set[Vertex] = set()
    queue: deque[Vertex] = deque([vertices[0]])
    adj = self.adjacency
    while queue:
        v = queue.popleft()
        if v in visited:
            continue
        visited.add(v)
        for to in adj[v]:
            if to not in visited:
                queue.append(to)

    return len(visited) == len(vertices)

def simple_paths_containing(self, target: Vertex) -> Dict[int, List[Tuple[Vertex, ...]]]:
    if target not in self.vertices:
        raise ValueError(f"Вершины {target!r} нет в графе")

    adj = self.adjacency
    paths: set[tuple[Vertex, ...]] = {(target,)}

    def canonical(path: list[Vertex]) -> tuple[Vertex, ...]:
        direct = tuple(path)
        reverse = tuple(reversed(path))
        return min(direct, reverse, key=lambda p: tuple(map(str, p)))

    def dfs(current: Vertex, path: list[Vertex]) -> None:
        if target in path and len(path) >= 2:
            paths.add(canonical(path))
        for nxt in adj[current]:
            if nxt in path:
                continue
            dfs(nxt, path + [nxt])

    for start in self.vertices:
        dfs(start, [start])

    grouped: dict[int, list[tuple[Vertex, ...]]] = defaultdict(list)
    for path in paths:
        grouped[len(path)].append(path)

    return {
        k: sorted(v, key=lambda p: (len(p), tuple(map(str, p))))
        for k, v in sorted(grouped.items())
    }

def producer_coefficients(self, target: Vertex) -> Dict[int, int]:
    grouped = self.simple_paths_containing(target)
    return {m: len(paths) for m, paths in grouped.items()}

def myerson_terms(self, target: Vertex) -> Dict[int, Fraction]:
    coeffs = self.producer_coefficients(target)
    return {
        vertices_count - 1: Fraction(count, vertices_count)
        for vertices_count, count in coeffs.items()
        if vertices_count >= 2
    }

def rooted_branch_producer_coefficients(self, target: Vertex) -> Dict[int, int]:
    if not self.is_tree():
        raise ValueError(
            "Рекурсия по ветвям реализована только для деревьев; "

```

```

        "для общего графа используйте перечисление простых путей."
    )
    if target not in self.vertices:
        raise ValueError(f"Вершины {target!r} нет в графе")

    adj = self.adjacency

    def poly_add(a: Dict[int, int], b: Dict[int, int]) -> Dict[int, int]:
        out: dict[int, int] = defaultdict(int)
        for k, v in a.items():
            out[k] += v
        for k, v in b.items():
            out[k] += v
        return dict(out)

    def poly_mul(a: Dict[int, int], b: Dict[int, int]) -> Dict[int, int]:
        out: dict[int, int] = defaultdict(int)
        for ka, va in a.items():
            for kb, vb in b.items():
                out[ka + kb] += va * vb
        return dict(out)

    def poly_shift_x(a: Dict[int, int]) -> Dict[int, int]:
        return {k + 1: v for k, v in a.items()}

    def branch(v: Vertex, parent: Vertex) -> Dict[int, int]:
        total = {0: 1}
        for child in adj[v]:
            if child == parent:
                continue
            total = poly_add(total, branch(child, v))
        return poly_shift_x(total)

    branches = [branch(neighbor, target) for neighbor in adj[target]]

    total = {0: 1}
    for b in branches:
        total = poly_add(total, b)
    for b1, b2 in combinations(branches, 2):
        total = poly_add(total, poly_mul(b1, b2))

    return poly_shift_x(total)

def draw_graph(
    graph: UndirectedGraph,
    target: Vertex | None = None,
    title: str = "Граф",
    save_path: str | None = None,
) -> None:

    G = nx.Graph()
    G.add_nodes_from(graph.vertices)
    G.add_edges_from(graph.edges)

    pos = nx.spring_layout(G, seed=42)

    node_colors = []
    for v in G.nodes:
        if v == target:
            node_colors.append("orange")
        else:
            node_colors.append("lightblue")

    plt.figure(figsize=(7, 5))

    nx.draw_networkx_edges(
        G,
        pos,
        width=2,
    )

    nx.draw_networkx_nodes(
        G,
        pos,
        node_color=node_colors,
        node_size=900,

```

```

        edgecolors="black",
    )

    nx.draw_networkx_labels(
        G,
        pos,
        font_size=12,
        font_weight="bold",
    )

    plt.title(title)
    plt.axis("off")
    plt.tight_layout()

    if save_path is not None:
        plt.savefig(save_path, dpi=200, bbox_inches="tight")
        print(f"Картинка графа сохранена: {save_path}")

    plt.show()

def print_recursive_phi_details(graph: UndirectedGraph, target: Vertex) -> None:
    if not graph.is_tree():
        print("Подробный рекурсивный расчет доступен только для дерева.")
        return

    if target not in graph.vertices:
        raise ValueError(f"Вершины {target!r} нет в графе")

    adj = graph.adjacency

    def poly_add(a: Dict[int, int], b: Dict[int, int]) -> Dict[int, int]:
        out: dict[int, int] = defaultdict(int)
        for k, v in a.items():
            out[k] += v
        for k, v in b.items():
            out[k] += v
        return dict(out)

    def poly_mul(a: Dict[int, int], b: Dict[int, int]) -> Dict[int, int]:
        out: dict[int, int] = defaultdict(int)
        for ka, va in a.items():
            for kb, vb in b.items():
                out[ka + kb] += va * vb
        return dict(out)

    def poly_shift_x(a: Dict[int, int]) -> Dict[int, int]:
        return {k + 1: v for k, v in a.items()}

    def poly_to_str(poly: Dict[int, int]) -> str:
        if not poly:
            return "0"

        parts: list[str] = []
        for power in sorted(poly):
            coef = poly[power]
            if coef == 0:
                continue

            if power == 0:
                part = str(coef)
            elif power == 1:
                part = "x" if coef == 1 else f"{coef}x"
            else:
                part = f"x^{power}" if coef == 1 else f"{coef}x^{power}"

            parts.append(part)

        return " + ".join(parts)

    print("\nПодробный рекурсивный расчет  $\phi$ :")
    print(f"Выбранная вершина p = {target}")

    branch_names: dict[Vertex, str] = {}
    def branch(v: Vertex, parent: Vertex) -> Dict[int, int]:
        child_polys: list[tuple[Vertex, Dict[int, int]]] = []

```

```

    for child in adj[v]:
        if child == parent:
            continue
        child_polys.append((child, branch(child, v)))

    total = {0: 1}
    for _, child_poly in child_polys:
        total = poly_add(total, child_poly)

    result = poly_shift_x(total)

    branch_names[v] = f"ψ_{v}"

    if child_polys:
        children_str = " + ".join(f"ψ_{child}" for child, _ in child_polys)
        print(
            f"    ψ_{v} = x(1 + {children_str}) = {poly_to_str(result)}"
        )
    else:
        print(f"    ψ_{v} = x = {poly_to_str(result)}")

    return result

branches: list[tuple[Vertex, Dict[int, int]]] = []
for neighbor in adj[target]:
    branches.append((neighbor, branch(neighbor, target)))

total = {0: 1}

for _, branch_poly in branches:
    total = poly_add(total, branch_poly)

for (_, b1), (_, b2) in combinations(branches, 2):
    total = poly_add(total, poly_mul(b1, b2))

phi = poly_shift_x(total)

branch_sum = " + ".join(f"ψ_{v}" for v, _ in branches)

pair_products = []
for (v1, _), (v2, _) in combinations(branches, 2):
    pair_products.append(f"ψ_{v1}ψ_{v2}")

if pair_products:
    formula_inside = "1 + " + branch_sum + " + " + " + ".join(pair_products)
else:
    formula_inside = "1 + " + branch_sum if branch_sum else "1"

print(f"\n    φ_{target}(x) = x({formula_inside})")
print(f"    φ_{target}(x) = {poly_to_str(phi)}")

def random_tree(n: int, seed: int = 2026) -> UndirectedGraph:
    if n < 1:
        raise ValueError("n должно быть положительным")

    rng = random.Random(seed)
    edges: list[tuple[int, int]] = []
    for v in range(2, n + 1):
        parent = rng.randint(1, v - 1)
        edges.append((parent, v))
    return UndirectedGraph(tuple(edges))

def print_vertex_analysis(graph: UndirectedGraph, target: Vertex, show_paths: bool = False) -> None:
    coeffs_enum = graph.producer_coefficients(target)
    terms = graph.myerson_terms(target)

    print("\n" + "-" * 92)
    print(f"Вершина p = {target}")
    print(format_producer(coeffs_enum, phi_name=f"φ_{target}"))

    print("Коэффициенты a_m при x^m, m = число вершин в пути:")
    print("    " + ", ".join(f"a_{m}={c}" for m, c in sorted(coeffs_enum.items())))
    edge_coeffs = {m - 1: c for m, c in coeffs_enum.items() if m >= 2}
    print("Коэффициенты A_k для путей длины k ребер, содержащих вершину p:")
    if edge_coeffs:

```

```

        print(" " + " ", ".join(f"A_{k}={c}" for k, c in sorted(edge_coeffs.items()))
    else:
        print(" нет")

    print(format_myerson(terms, target))
    if graph.is_tree() and show_paths:
        print_recursive_phi_details(graph, target)

    if graph.is_tree():
        coeffs_rec = graph.rooted_branch_producer_coefficients(target)
        result = "совпало" if coeffs_rec == coeffs_enum else "НЕ совпало"
        print(f"Проверка через рекурсию по ветвям: {result}")

    if show_paths:
        grouped_paths = graph.simple_paths_containing(target)
        print(f"Пути, содержащие вершину {target}:")
        for vertices_count, paths in grouped_paths.items():
            length_edges = vertices_count - 1
            readable = ["-".join(map(str, path)) for path in paths]
            print(f"    длина {length_edges} ребер ({vertices_count} вершин): {len(paths)} шт. ->
{readable}")

def solve_graph_task() -> None:
    lecture_graph = UndirectedGraph(edges=((2, 4), (2, 5), (2, 1), (1, 3), (3, 6)))
    targets = [1, 2, 3, 4, 5, 6]
    draw_graph(
        lecture_graph,
        target=2,
        title="Граф из лекции. Выделена вершина 2",
        save_path="data/lecture_graph_vertex_2.png",
    )

    print("\nГраф из лекции")
    print(f"Ребра: {lecture_graph.edges}")
    print(f"Вершины: {lecture_graph.vertices}")
    print(f"Является деревом: {lecture_graph.is_tree()}")

    for target in targets:
        print_vertex_analysis(lecture_graph, target, show_paths=(target == 2))

    print("\n" + "-" * 92)
    print("Тест: дерево на 8 вершинах")

    test_graph = UndirectedGraph(edges=((1, 2), (2, 3), (3, 4), (1, 5), (2, 6), (5, 7), (5, 8)))
    test_targets = [1, 2, 3, 4, 5, 6, 7, 8]
    draw_graph(
        test_graph,
        target=1,
        title="Тестовый граф. Выделена вершина 1",
        save_path="data/test_graph_vertex_1.png",
    )

    print(f"Является деревом: {test_graph.is_tree()}")

    for target in test_targets:
        print_vertex_analysis(test_graph, target, show_paths=(target == 1))

    print("\n" + "-" * 92)
    print("Случайный тест: дерево на 8 вершинах")
    test_graph = random_tree(n=8, seed=2026)
    print(f"Ребра: {test_graph.edges}")
    for target in test_graph.vertices:
        coeffs = test_graph.producer_coefficients(target)
        terms = test_graph.myerson_terms(target)
        print(f"p={target}: {format_producer(coeffs, phi_name=f'φ_{target}')}" ;
{format_myerson(terms, target)})

    draw_graph(
        test_graph,
        target=1,
        title="Случайное дерево. Выделена вершина 1",
        save_path="data/random_tree_vertex_1.png",
    )

if __name__ == "__main__":
    solve_graph_task()

```